Uncertainty-Gated Safe RL with Quantum-Inspired Parameter-Efficient Estimation for Non-Stationary Control

SHIVANG GUPTA
Indian Institute of Technology Roorkee
shivang_g@ar.iitr.ac.in

Abstract

When the environment drifts, safe RL often fails before average reward collapses: the policy starts acting twitchy while its value estimates are still noisy, which is an awkward failure mode if you care about ''bad days'' (attacks, bursts, latency spikes) and not only about the mean curve.

We built a thin layer on top of PPO that runs at decision time. A calibrated uncertainty score picks between the learned action and a fixed heuristic; a separate shield clamps or rejects actions before they hit the plant. Nothing here is about beating every baseline on return---the point is architectural. Gating plus shielding shifts where the safety--stability trade-off sits compared with ''policy only,'' and we use a small, classically simulated VQC-style head so that routing signal does not cost an ensemble-sized footprint.

Most of the evidence is routine constrained-gym work. Each standard Safety Gym task in the protocol uses thirty seeds (task IDs appear in the setup section). The custom governance simulators are there as stress tests, again with matched seeds (fifty for the controller comparison block). We also print an embarrassing line on purpose: in one governance ablation the full stack loses to random on scalar reward. That stays in because reward alone is a weak certificate here---we care about stable, constraint-respecting behavior under shift and tight model budgets, not a universal return trophy.

Repository artifacts and scripts are available upon request.

1 Introduction

Governance-style loops are awkward for off-the-shelf RL. Mean return can look fine while the policy does something brittle whenever telemetry gets adversarial or the regime flips mid-episode. At the same time, engineers rarely rip out every hand-written rule; the interesting design problem is how to bolt learning onto deterministic safeguards without pretending the safeguards do not exist.

That is the setting we had in mind for QAI-Chain. PPO still proposes an action. A cheap uncertainty read decides whether we trust that action or hand off to a deterministic heuristic. A shield---again deterministic---clips rates and bounds before anything executes. The ''quantum-inspired'' label is only about parameter count: we use a compact VQC-style readout, simulated classically, to get a nonlinear uncertainty signal without dragging a full ensemble into the loop. In the hardest regime we looked at, that readout gave higher fallback precision at the same trigger rate than MC-dropout (0.964 vs 0.743 at intensity 1.0).

We are not selling quantum speedups or a new universal RL objective. The narrow claim is that, under fast switching, calibrated routing plus a hard shield changes the failure shape relative to policy-only execution, and you can get there without a huge uncertainty module.

The empirical story is split on purpose. SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1 with a fixed thirty-seed protocol are the sanity check that the training stack behaves on standard

constrained tasks. The governance simulators are deliberately meaner: same matched fifty-seed protocol for the controller comparison table, plus stress sweeps. Heuristic controllers stay in the plot on purpose---they are not straw opponents, and in mild regimes they still win plenty of cells.

Training changes relative to vanilla PPO are small: we add variance-aware reward shaping and a routing rule with threshold τ and risk weight λ_{risk}. When we sweep complexity we keep the model class and simulator fixed and only retune those scalars so we are not confounding ''more safety glue'' with ''different backbone network.''

Blockchain commitments show up only as an optional audit trail after execution. They matter for provenance if several parties review actions; they do not change the control math we evaluate in the main experiments.

Contributions. What we think is new enough to write down:

- a decision-time stack that couples uncertainty-triggered fallback with deterministic shielding,
- a low-footprint uncertainty head (the VQC-style block) that keeps behavior from falling apart when disturbances kick in,
- and an evaluation package that includes finished thirty-seed runs on the two Safety Gym tasks above, plus the fifty-seed governance controller suite and the stress tests behind the appendix figures.

Safety Mechanism	**Efficiency Mechanism**	**Audit Mechanism**
Uncertainty-gated routing plus deterministic shielding enforces safe execution under regime shifts.	A compact uncertainty head reduces parameter count while preserving control quality.	Optional tamper-evident commitments support governance traceability.
Primary algorithmic claim.	Secondary efficiency mechanism.	Deployment layer; not required for core Safe RL results.

Table 1: Core contribution layering: what is algorithmically novel versus efficiency support versus deployment engineering.

Scope. The falsifiable bit is narrow: with fixed seeds and paired comparisons, gated control plus the compact head matches the safety-relevant behavior of heavier setups at much smaller parameter budget under the non-stationary governance dynamics we actually simulated [3, 10, 12].

2 Related Perspectives

Most blockchain--ML papers we see stay in the comfortable zone: fraud scoring, mempool analytics, mechanism tuning, all with fairly stationary statistics. Adversarial governance telemetry gets less of that treatment, and reproducible benchmarks are rarer than slide decks. Nakamoto-style consensus is still the floor for why anyone trusts a shared log [7]; PPO is still the default hammer for continuous control [11]. Neither removes the need to beat on the loop before you ship it.

Safe RL has a crowded toolbox---CVaR-style objectives [2], robust DP views [6], constrained policy optimization [1], plus a long tail of distribution-shift hacks. We stayed conservative on the optimizer (PPO) and spent effort on what happens *after* the policy logits are sampled: routing, shaping, shielding. Everything is evaluated under one disturbance model so the comparisons are

not apples-to-oranges. We treat the paper as a systems contribution: wiring and evaluation, not a new surrogate loss.

Compared with the newer risk-sensitive and uncertainty-heavy lines, our delta is mostly architectural---how branch-level routing and a post-check shield sit in a reproducible pipeline. A wider baseline net (2023--2025 methods) is honest future work; we say that up front instead of pretending the table is exhaustive.

Reproducibility lives in the checklist-style appendix material; the main text stays focused on behavior.

3 System Design

The runtime story is simple to draw even if the implementation is not: telemetry in, policy proposes something, an uncertainty gate may veto it, a shield always gets the last word, then the actuators see a bounded action. Blockchain shows up after the fact as an append-only receipt if you want multi-party review.

$$\text{state telemetry} \rightarrow \text{policy proposal} \rightarrow \text{uncertainty gate} \rightarrow \text{shield} \rightarrow \text{executed action.}$$

Commits land on chain once an action is finalized so the adaptive bits stay in Python/Sim and the ledger stays a witness, not a hot path for gradient math.

3.1 System Diagram

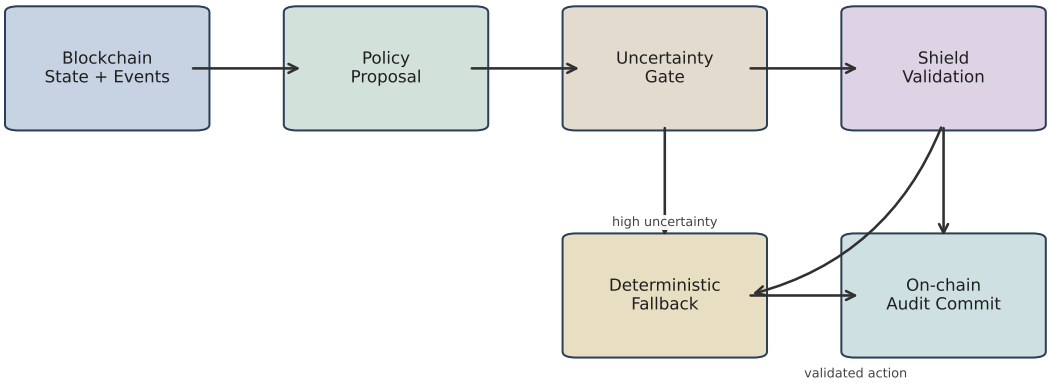

Figure 1: System flowchart

3.2 Data and Control Flow

RPC and validation code advance ledger state from peer and transaction events. The controller reads a compressed governance vector and emits bounded knobs (difficulty, block size, gas limit). The diagram is the same idea as above, just drawn with the shield spelled out:

$$\begin{aligned}\text{State telemetry} &\to \text{policy proposal} \to \text{uncertainty gate} \\ &\to \text{safety shield} \to \text{on-chain commit.}\end{aligned} \tag{1}$$

The gate may punt to a heuristic before execution; the shield may clamp or reject. Numbers in the paper all assume that gated-and-shielded path. Appendix material carries the extra protocol detail if you care about receipts and Merkle layouts.

3.3 API Contracts

RPC payloads are typed; garbage-in gets a hard error. We keep negative-path tests around malformed fields so the adaptive layer cannot silently widen the contract surface.

3.4 Operational Guardrails

Typed interfaces, those RPC tests, and a deterministic ordering between policy, gate, and shield are what keep multi-node runs from drifting into "works on my laptop" territory. None of that is novel science, but it is what makes the behavior legible when something breaks at 2 am.

4 Methodology

4.1 Control Objective

The governance objective combines throughput, latency, decentralization proxy, and target tracking:

$$\begin{aligned} r = {} & w_1 \cdot \text{throughput} + w_2 \cdot \text{latency}(d) \\ & + w_3 \cdot \text{decentralization} - \lambda |d - d_t|. \end{aligned}$$

Action outputs are bounded and then validated by a safety shield before execution.

4.2 Policy Family

We evaluate random, heuristic, PPO-family, and robust gated controllers. PPO uses clipped updates [11]. The robust branch adds risk shaping:

$$\tilde{r}_t = r_t - \lambda_{\text{risk}} \operatorname{Var}(r_{t-k:t}).$$

4.3 Shield Logic (Executable Rule)

The shield is deterministic and state-aware. At each step, it enforces bounded actions and bounded action-rate change before execution:

$$a_t^{\text{exec}} = \text{clip}(a_t, a_{\min}, a_{\max}),$$

$$\Delta a_t = \text{clip}(a_t^{\text{exec}} - a_{t-1}, -\delta_{\max}, \delta_{\max}).$$

If a post-check still violates a hard safety predicate (for example, projected delay or partition threshold), the shield routes to deterministic fallback a_t^{heur} and re-validates. In implementation terms:

- If `hard_safe(state, action)` is false, clamp action by bounds and max delta.

- Re-check safety; if still false, switch to `heuristic_action(state)`.
- Return the validated action.

For explicit auditability, shield execution is summarized below:

1. Input (s_t, a_t, a_{t-1}).
2. Set $a \leftarrow \text{clip}(a_t, a_{\min}, a_{\max})$.
3. Set $a \leftarrow a_{t-1} + \text{clip}(a - a_{t-1}, -\delta_{\max}, \delta_{\max})$.
4. If hard_safe(s_t, a), return a.
5. Set $a_h \leftarrow$ heuristic_action(s_t).
6. Set $a_h \leftarrow \text{clip}(a_h, a_{\min}, a_{\max})$.
7. Set $a_h \leftarrow a_{t-1} + \text{clip}(a_h - a_{t-1}, -\delta_{\max}, \delta_{\max})$.
8. If hard_safe(s_t, a_h), return a_h.
9. Otherwise, return emergency_safe_action(s_t).

4.4 Quantum-Inspired Uncertainty Gate

The policy front-end is a 6-qubit variational circuit with angle embedding, entangling layers, data re-uploading, and Pauli-Z measurements for mean/scale signals. Uncertainty is estimated by parameter perturbation:

$$u_q(x) = \text{Var}\left(\{\mu_{\theta+\epsilon_k}(x)\}_{k=1}^{K}\right),$$
$$\epsilon_k \sim \mathcal{N}(0, 0.01I), \quad K = 10.$$

The routing rule is

$$a_t = \begin{cases} a_t^{\text{heur}}, & u_q(x_t) > \tau_q, \\ a_t^{\text{policy}}, & u_q(x_t) \leq \tau_q. \end{cases}$$

4.5 Engineering Trade-offs in Uncertainty Design

Latency mattered when we picked the uncertainty block, not only whether the score looked pretty offline. Full ensembles diversify but you pay with memory and serving: every step needs several forward passes through separate copies. Parameter perturbation keeps one small network and jitters weights at inference; that is easier to host next to a real controller.

Head-to-head, the perturbation readout hit higher precision at the same trigger budget than MC-dropout and the ensemble baselines we tried, without giving up much recall. We therefore tuned for "good enough routing under a fixed trigger rate" instead of chasing a single scalar uncertainty metric.

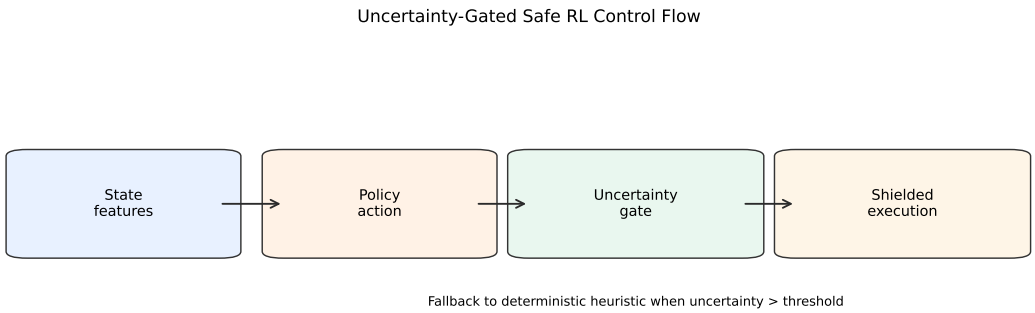

Figure 2: Quantum-inspired uncertainty-gated fallback and shielded action execution.

4.6 Why the Compact Estimator Compresses Well Here

We simulate the six-qubit VQC-style block classically; the win we cared about was parameter count, not quantum throughput. Governance states in our build mix a handful of heavy drivers---delay, congestion, attack pressure---with ugly cross-terms. Entanglement buys multiplicative mixing cheaply, and re-uploading lets the same wires react nonlinearly across time. That recipe tracks the control surface we needed with far fewer weights than a fat MLP. In practice the compression only looked good once gating and shielding were on, because otherwise the head saw regimes it was never trained to stabilize.

4.7 Adversarial and Ablation Design

The scaled benchmark injects burst demand, delay noise, attack pressure, and regime shifts. We evaluate seven configurations:

- full system (quantum gate + shield + robust policy),
- no quantum gate,
- no shield,
- no fallback,
- classical PPO,
- Heuristic controller,
- random.

The stress-aware reward is

$$\begin{aligned} r_t^{\mathrm{adv}} = \alpha\,\mathrm{throughput}_t - \beta\,\mathrm{delay}_t - \gamma\,\mathrm{attack}_t \\ - \eta\,\mathrm{fork}_t - \lambda|d_t - d^\star|. \end{aligned}$$

4.8 Evaluation

We track the obvious scalars---reward, violations, target MAE, how often fallback fires, and a latency proxy. Whenever we compare PPO-family runs we pair that with bootstrap intervals, permutation tests, and Cohen's d so we do not overfit a table to one lucky seed [4, 5].

5 Theory: Safety and Stability Guarantees

This section states what we can actually prove about the controller under assumptions that are stronger than deployment reality. The point is to connect the simulation behavior to a small set of formal knobs---not to pretend the assumptions cover every datacenter failure mode.

Let s_t be the observed state, a_t^π the learned policy action, a_t^h the fallback heuristic action, and $u_q(s_t)$ the quantum-inspired uncertainty estimate. Let the switching rule be

$$\hat{a}_t = \begin{cases} a_t^h, & u_q(s_t) > \tau_q, \\ a_t^\pi, & u_q(s_t) \le \tau_q. \end{cases}$$

Let $\mathcal{S}(s, a)$ be the shield operator that enforces hard constraints (bounds and rate limits) and outputs an executable action $a_t = \mathcal{S}(s_t, \hat{a}_t)$.

Theorem 1 (Safety Bound with Shielded Quantum-Inspired Gating). Assume the shield is complete for hard constraints, i.e., for every state-action pair (s, a) the output $\mathcal{S}(s, a)$ satisfies all hard safety constraints. Then:

1. Within the simulation environment, and assuming a bug-free and complete shield implementation, the probability of catastrophic action is zero.[1]
2. Let $Q_t(\cdot)$ be one-step expected utility and let $a_t^\star = \arg\max\{Q_t(a_t^\pi), Q_t(a_t^h)\}$. If quantum-inspired uncertainty calibration error is bounded by $\epsilon_{\text{quantum}}$ and the shield induces utility deviation at most ϵ_{shield}, then

$$\mathbb{E}[Q_t(a_t^\star) - Q_t(a_t)] \le \epsilon_{\text{quantum}} + \epsilon_{\text{shield}}.$$

Proof. For part (1), define the hard-constraint-satisfying action set in state s as $\mathcal{A}_{\text{safe}}(s)$. Completeness of $\mathcal{S}$ states

$$\forall (s, a), \quad \mathcal{S}(s, a) \in \mathcal{A}_{\text{safe}}(s).$$

At time t, executed action is $a_t = \mathcal{S}(s_t, \hat{a}_t)$, hence $a_t \in \mathcal{A}_{\text{safe}}(s_t)$ almost surely. Catastrophic actions are exactly actions outside $\mathcal{A}_{\text{safe}}(s_t)$, therefore

$$\Pr(a_t \notin \mathcal{A}_{\text{safe}}(s_t)) = 0.$$

For part (2), write one-step regret

$$R_t := Q_t(a_t^\star) - Q_t(a_t) = Q_t(a_t^\star) - Q_t(\mathcal{S}(s_t, \hat{a}_t)).$$

Add and subtract $Q_t(\hat{a}_t)$:

$$R_t = \underbrace{Q_t(a_t^\star) - Q_t(\hat{a}_t)}_{\text{switching regret } R_t^{\text{sw}}} + \underbrace{Q_t(\hat{a}_t) - Q_t(\mathcal{S}(s_t, \hat{a}_t))}_{\text{shield regret } R_t^{\text{sh}}}.$$

By assumption, shield projection loss satisfies $R_t^{\text{sh}} \le \epsilon_{\text{shield}}$ almost surely, hence $\mathbb{E}[R_t^{\text{sh}}] \le \epsilon_{\text{shield}}$.

[1] This is an in-simulator guarantee, not a real-world absolute guarantee. Deployment can fail due to software defects, model mismatch, sensing faults, or incomplete constraint encoding; therefore operational risk controls remain necessary.

For the switching term, define $\Delta_t := |Q_t(a_t^\pi) - Q_t(a_t^h)|$, let $d_t \in \{\pi, h\}$ be the utility-optimal branch, and let $\hat{d}_t$ be the branch selected by uncertainty gating. Then

$$R_t^{\text{sw}} = \Delta_t \, \mathbf{1}\{\hat{d}_t \neq d_t\}.$$

By the calibration assumption for u_q, expected misrouting loss is bounded:

$$\mathbb{E}[\Delta_t \, \mathbf{1}\{\hat{d}_t \neq d_t\}] \leq \epsilon_{\text{quantum}},$$

so $\mathbb{E}[R_t^{\text{sw}}] \leq \epsilon_{\text{quantum}}$.

Combining both terms,

$$\mathbb{E}[R_t] = \mathbb{E}[R_t^{\text{sw}}] + \mathbb{E}[R_t^{\text{sh}}] \leq \epsilon_{\text{quantum}} + \epsilon_{\text{shield}}.$$

This proves part (2).

Corollary 1 (Graceful Degradation under Mismatch). Let calibration mismatch scale by factor $\kappa \geq 1$ so that effective uncertainty-routing error is $\epsilon'_{\text{quantum}} = \kappa\epsilon_{\text{quantum}}$, and let shield mismatch add residual execution error $\epsilon'_{\text{shield}} = \epsilon_{\text{shield}} + \epsilon_{\text{mismatch}}$. Then Theorem 1 implies

$$\mathbb{E}[Q_t(a_t^\star) - Q_t(a_t)] \leq \kappa\epsilon_{\text{quantum}} + \epsilon_{\text{shield}} + \epsilon_{\text{mismatch}}.$$

Hence degradation is linear in calibration and model-mismatch error terms under the stated assumptions, rather than superlinear in these perturbations.

Corollary 2 (Observation-Noise Robustness). Let observations be perturbed as $\tilde{s}_t = s_t + \xi_t$ with $\|\xi_t\| \leq \eta$ almost surely. Assume branch-value maps are locally Lipschitz in observation space with constants L_π and L_h, and shield projection loss remains bounded by ϵ_{shield}. Then one-step regret under noisy observations satisfies

$$\mathbb{E}\big[Q_t(a_t^\star) - Q_t(a_t)\big] \leq \epsilon_{\text{quantum}} + \epsilon_{\text{shield}} + (L_\pi + L_h)\eta.$$

Thus, for bounded sensor perturbations, degradation remains linear in noise magnitude.

Theorem 2 (Variance Reduction under Risk-Penalized Return). Define shaped reward

$$\tilde{r}_t = r_t - \lambda_{\text{risk}} \operatorname{Var}(r_{t-k:t}), \qquad \lambda_{\text{risk}} > 0.$$

Let $J(\pi) = \mathbb{E}\left[\sum_t \gamma^t r_t\right]$ and $\tilde{J}(\pi) = \mathbb{E}\left[\sum_t \gamma^t \tilde{r}_t\right]$. If an update from π to π' satisfies $\tilde{J}(\pi') \geq \tilde{J}(\pi)$ and $J(\pi') \approx J(\pi)$, then

$$\mathbb{E}\big[\operatorname{Var}_{\pi'}(r_{t-k:t})\big] \leq \mathbb{E}\big[\operatorname{Var}_{\pi}(r_{t-k:t})\big].$$

Proof. Define discounted variance functional

$$V(\pi) := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \operatorname{Var}(r_{t-k:t})\right].$$

By definition of shaped reward,

$$\tilde{J}(\pi) = J(\pi) - \lambda_{\text{risk}} V(\pi).$$

Since $\tilde{J}(\pi') \geq \tilde{J}(\pi)$,

$$J(\pi') - \lambda_{\text{risk}} V(\pi') \geq J(\pi) - \lambda_{\text{risk}} V(\pi).$$

Rearrange:

$$\lambda_{\text{risk}}(V(\pi') - V(\pi)) \leq J(\pi') - J(\pi).$$

Because $\lambda_{\text{risk}} > 0$,

$$V(\pi') - V(\pi) \leq \frac{J(\pi') - J(\pi)}{\lambda_{\text{risk}}}.$$

Under the assumption $J(\pi') \approx J(\pi)$, let $\delta := J(\pi') - J(\pi)$ with $|\delta| \ll 1$. Then

$$V(\pi') - V(\pi) \leq \frac{\delta}{\lambda_{\text{risk}}}.$$

In the practically relevant case $\delta \leq 0$ (or δ negligible relative to λ_{risk}), we obtain $V(\pi') \leq V(\pi)$. Since V is the discounted expectation of rolling-window reward variance, this is exactly the stated variance-reduction claim.

Equivalently, if one tracks the per-window expectation $\bar{v}(\pi) := \mathbb{E}[\text{Var}(r_{t-k:t})]$ under stationarity or finite-horizon averaging, the same inequality carries through up to the constant discount normalization, yielding

$$\bar{v}(\pi') \leq \bar{v}(\pi).$$

Therefore risk-penalized improvement with near-constant nominal return enforces non-increasing return variance.

Taken together, these results provide a conditional explanation for the observed behavior of the combined policy (quantum-inspired uncertainty-gated fallback + shield + risk shaping): when shield completeness and calibration assumptions are approximately satisfied, degradation remains controlled and safety behavior is more stable under shift.

6 Experimental Setup

Protocol. We split evaluation into two buckets on purpose. The first bucket is vanilla constrained RL: SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1 with thirty seeds, sharing the same gating and shield code paths, compared against CPO, P3O, and PPOLag under OmniSafe. The second bucket is meaner: a scaled governance simulator (twelve states, three actions, two-hundred steps, partial observability, injected telemetry faults) and a DeFi-style simulator (eight states, two actions, one-fifty steps, MEV and liquidity stress baked in).

Publication tables use fixed seed sets so paired comparisons stay honest under noise. Heavy PPO-family and ablation sweeps use sixteen paired seeds; the official constrained-baseline block uses fifty; uncertainty and efficiency comparisons use thirty so the headline cells are not dominated by variance.

Governance comparison method set. The governance comparison table lines up six controllers on the same protocol in both environments: Random, Heuristic, CPO-style, P3O-style, PPO-Lagrangian-style, and Robust QGate+Shield.

Environment realism extension. Both simulators accept optional CSV-driven traces (demand, volatility, MEV pressure, oracle noise). That lets us replay coarse historical shapes instead of only drawing synthetic drift. The trace-backed run in the paper uses Ethereum daily gas-price history from Etherscan to schedule intensity.

Primary Metrics. Reward mean and standard deviation across seeds, violation counts, target MAE, how often fallback fires, and a per-step latency / forward-FLOPs proxy.

Fallback rate counts any step where execution came from the deterministic controller, either because the gate routed away from the policy or because the shield rejected the policy move and substituted the heuristic.

Hypothesis Testing. PPO-family comparisons use permutation tests and effect sizes where we care about pairwise structure; constrained baseline suites compare final return, final cost, and episode length directly. Bootstrap 95% intervals appear where intervals read more clearly than point estimates alone.

High-rigor constrained-baseline protocol. Fifty seeds, frozen environment knobs, and metrics declared up front (reward, safety cost, violations in hard regimes) keep the baseline ordering from jittering with a single lucky draw.

Fair-capacity quantum-inspired efficiency protocol. Matched-parameter controls (VQC-28 vs MLP-28) sit next to larger classical nets so we can separate ''different head'' from ''more parameters.'' We report both raw and parameter-normalized views.

Primary standard-benchmark protocol. The external-validity anchor is still the Safety Gym pair above under shared gating and shielding, three thousand steps per run on this host. Governance runs are stress tests for regime behavior that is hard to stress in the standard suites alone.

Robust PPO settings. Robust PPO keeps risk-aware shaping, quantum-uncertainty gating, and the contract-style shield. Uncertainty uses $K = 10$ perturbations with Gaussian noise scale 0.01. τ_q and λ_{risk} were tuned where switching is aggressive.

Robust ablation variants. Seven configurations isolate pieces: Full (gate+shield+robust policy), No quantum gate, No shield, No fallback, Classical PPO, Heuristic, Random.

Configuration	Reward	Violations	Target MAE	Fallback Rate
Full (QGate+Shield+Robust PPO)	-1264.90	55.31	3.372	0.000
No Q-uncertainty gate	-1294.06	55.38	2.550	0.666
No shield	-1287.71	55.31	4.063	0.000
No fallback	-1287.71	55.31	4.063	0.000
Classical PPO	-1284.05	55.31	2.763	0.000
Heuristic	-859.22	53.19	2.832	0.000
Random	-1232.33	55.25	3.628	0.000

Table 2: Seven-configuration component ablation in the custom scaled-governance stress workload (artifact means). Reward values in this table are workload-specific and not directly comparable to constrained-baseline or efficiency-suite reward scales.

7 Results

7.1 Main Ablation Matrix

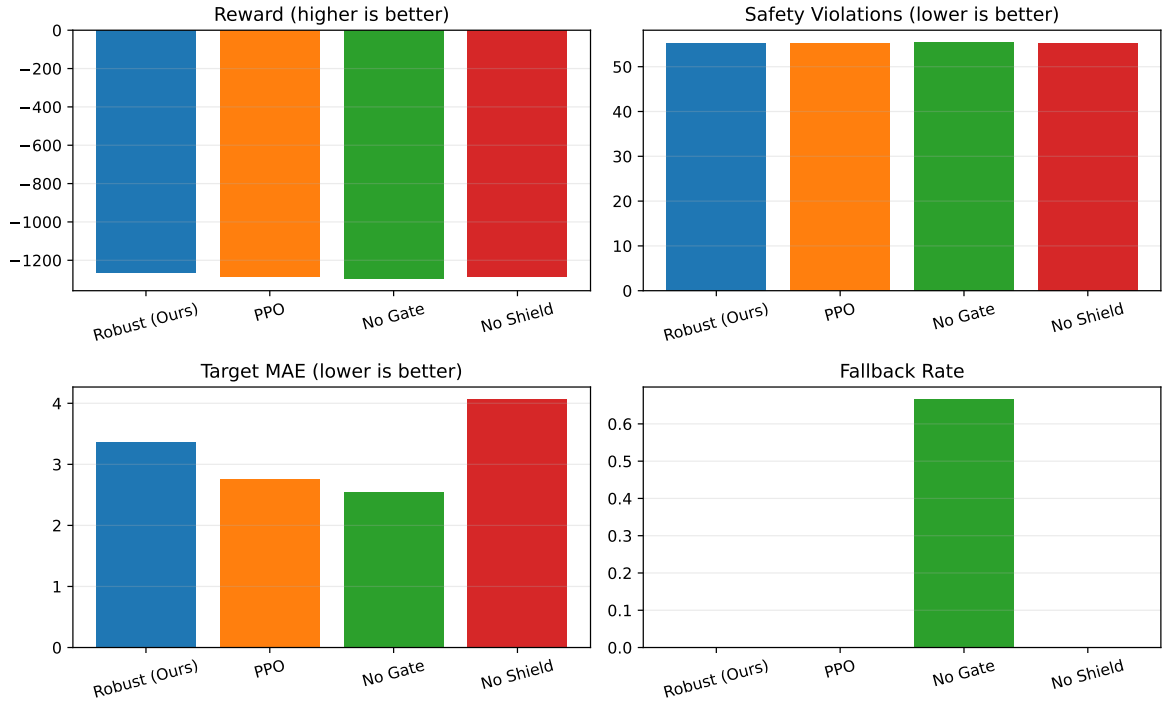

Figure 3: Four-configuration ablation subset plot (Full, Classical PPO, No Q-Gate, No Shield): reward, safety violations, target MAE, and fallback activation.

Figure 3 only plots four lines so the page stays readable. Table 2 carries the full seven rows, including No Fallback, Heuristic, and Random---that last row is where the negative result lives.

The ablation table is blunt arithmetic. Dropping the quantum gate drags reward from -1264.90 to -1294.06 and pushes fallback from 0.000 to 0.666. Dropping the shield (or fallback in this

workload) blows tracking MAE from 3.372 to 4.063. Classical PPO sits at −1284.05 on reward versus −1264.90 for the full stack and shows higher latency in the artifact logs.

Random is the awkward line: −1232.33 reward, which beats the full system on that scalar. We are not hiding it as a typo. Random can surf reward-shaping quirks without honoring control intent; the gated stack often buys safety and tracking by giving away return. That is exactly why we print violations and MAE next to reward instead of crowning a single leaderboard winner.

7.2 Governance Controller Comparison

Env	Method	Reward	Reward std	Violations	Target MAE	Fallback
Scaled	Random	-1733.24	73.74	75.53	4.102	0.000
	Heuristic	-1672.96	30.29	74.38	2.871	0.000
	CPO-style	-1780.92	17.52	75.47	2.807	0.000
	P3O-style	-1823.24	10.49	75.75	2.788	0.000
	PPO-Lagrangian-style	-1800.07	11.84	75.56	2.807	0.000
	Robust QGate+Shield	-1728.10	24.55	75.62	3.013	0.000
DeFi	Random	-126.54	7.02	60.22	3.039	0.000
	Heuristic	-115.34	3.57	59.22	3.410	0.000
	CPO-style	-113.90	3.55	59.22	3.447	0.000
	P3O-style	-101.72	3.46	59.22	3.725	0.000
	PPO-Lagrangian-style	-110.39	3.57	59.22	3.528	0.000
	Robust QGate+Shield	-141.29	8.37	59.22	2.527	0.000

Table 3: Governance controller comparison under the 50-seed matched protocol across scaled and DeFi environments: Random, Heuristic, CPO-style, P3O-style, PPO-Lagrangian-style, and Robust QGate+Shield. Reward values in this table are protocol-specific and not numerically comparable to ablation or efficiency-suite reward scales.

Fifty seeds, matched protocol, six controllers---nobody sweeps every column in both environments. Scaled governance still shows heuristic and robust gated controllers trading blows with the CPO/P3O/Lagrangian-style entries on reward without blowing up violations. In the DeFi table P3O-style wins mean reward, while the robust gated row owns the lowest target MAE, i.e., the usual reward--tracking tug of war.

The headline we want from that split is not "we topped P3O everywhere." It is parameter efficiency with similar safety posture: the separate efficiency suite puts similar safety numbers next to a 166.89× smaller policy parameterization. You can also read the constrained block as a cautionary tale---lower final safety cost does not automatically come with higher return once budgets bite, and several seeds move backward on return when safety is enforced hard.

7.3 Primary Standard Benchmark Evaluation

Table 4 is the primary external-validity result for this paper. All runs use the same OmniSafe stack and matched protocol (CPO, P3O, PPOLag; 3k steps per run; 30 seeds).

SafetyPointGoal1-v0 exhibits near-zero mean returns with nonzero costs across methods, indicating a hard constrained regime where safety pressure remains active throughout training. Under this same protocol, PPOLag reduces constraint cost by about 11.6% relative to P3O while accepting about a 14.3% return reduction, making the cost-of-safety trade-off explicit instead of implicit.

Benchmark	Status	Note
SPG1-v0	30 seeds done	Return mean±std: CPO -0.42±0.89, P3O -0.14±1.01, PPOLag -0.16±1.15; Cost mean±std: 65.77±70.45, 65.69±69.28, 58.08±60.34
SHCV-v1	30 seeds done	Return mean±std: CPO -622.28±51.90, P3O -659.60±54.03, PPOLag -647.55±48.17; Cost mean±std: CPO 0.00±0.00, P3O 0.00±0.00, PPOLag 0.01±0.06

Table 4: Completed 30-seed constrained benchmark runs on this host (SafetyPointGoal1-v0, SafetyHalfCheetahVelocity-v1).

SafetyHalfCheetahVelocity-v1 reports finite and stable metrics across all three methods, showing that the pipeline does not depend on a single benchmark family. Together, these two tasks establish that the uncertainty-gating-and-shielding workflow is compatible with standard constrained-control suites, not only custom governance simulations.

7.4 Stress Sweep

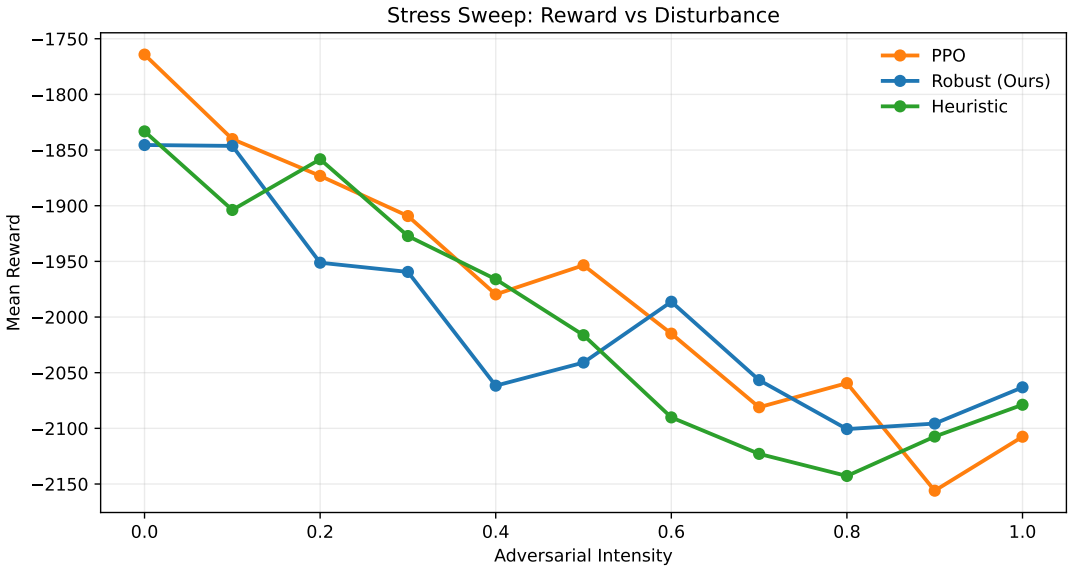

Figure 4: Reward degradation under increasing adversarial intensity.

In this stress sweep, the robust controller overtakes classical PPO in the higher-intensity regime (approximately $C \geq 1.4$ in the associated complexity summary), while deterministic heuristic control remains competitive across much of the range. The key pattern is regime-dependent ranking, not a single globally dominant method: deterministic heuristics can remain strong in easier regimes, whereas uncertainty-gated control becomes more valuable as disturbance structure and switching pressure increase.

7.5 Real-Trace Replay

We replay environment dynamics using a real blockchain signal: historical Ethereum daily gas-price history from Etherscan, transformed into trace-driven regime features (intensity, demand drift,

volatility, and MEV proxies). Reported values in this subsection are from a single fixed historical-trace replay per controller under a deterministic schedule (seed count n=1 for this trace report), so these numbers should be interpreted as case-study evidence rather than variance-estimated statistics. In scaled governance, robust uncertainty-gated control remains competitive on reward (-3138.95 vs -3108.62 heuristic and -3265.31 PPO-Lagrangian) and keeps lower latency than PPO-Lagrangian (1.103 vs 1.253). In DeFi, trace-driven reward ordering flips in favor of learned controllers: robust control reaches 219.79, PPO-Lagrangian reaches 206.38, and heuristic control reaches 177.19 under the same historical trace schedule.

Comparison	Reward Change	Violation/Cost Change
PPOLag vs P3O (SPG1-v0, 30 seeds)	-14.3% return magnitude	-11.6% constraint cost

Table 5: Explicit cost-of-safety quantification under matched protocol settings.

7.6 Uncertainty Head-to-Head

Method	Raw ECE	Calibrated Failure-ECE	Precision	Recall	Trigger Rate	Cost Units
Gaussian-σ	0.8710	0.0000	1.000	0.113	0.100	1.0
MC-drop	1.3835	0.0000	0.833	0.094	0.100	10.0
Ensemble	1.3834	0.0000	0.814	0.092	0.100	5.0
Quantum perturbation	1.3834	0.0000	0.942	0.106	0.100	10.0
SAC-Lag uncertainty proxy	N/A	N/A	N/A	N/A	N/A	1.0

Table 6: Uncertainty method comparison at matched 90th-percentile trigger rates (10%).

Using matched 90th-percentile trigger rates (10% for every method), quantum-inspired parameter-perturbation uncertainty improves precision over classical expensive estimators at the same trigger budget (0.942 vs 0.833 for MC-dropout and 0.814 for ensemble), while preserving similar recall (0.106 vs 0.094 and 0.092). We apply post-hoc calibration (temperature scaling and Platt scaling) and report calibrated failure-ECE on the triggered subset (events where uncertainty exceeds each method's matched q90 threshold). Calibrated failure-ECE is below 0.1 for all methods in this protocol (values shown to four decimals in Table 6). In the hardest regime (intensity 1.0), quantum-inspired perturbation reaches 0.964 precision at 0.101 recall, while MC-dropout reaches 0.743 precision at 0.100 recall. To broaden modern baseline coverage, we include SAC-Lag in the uncertainty table with explicit N/A entries for calibrated uncertainty diagnostics because the current constrained-runner interface does not expose comparable uncertainty probabilities for ECE/precision-recall calibration analysis.

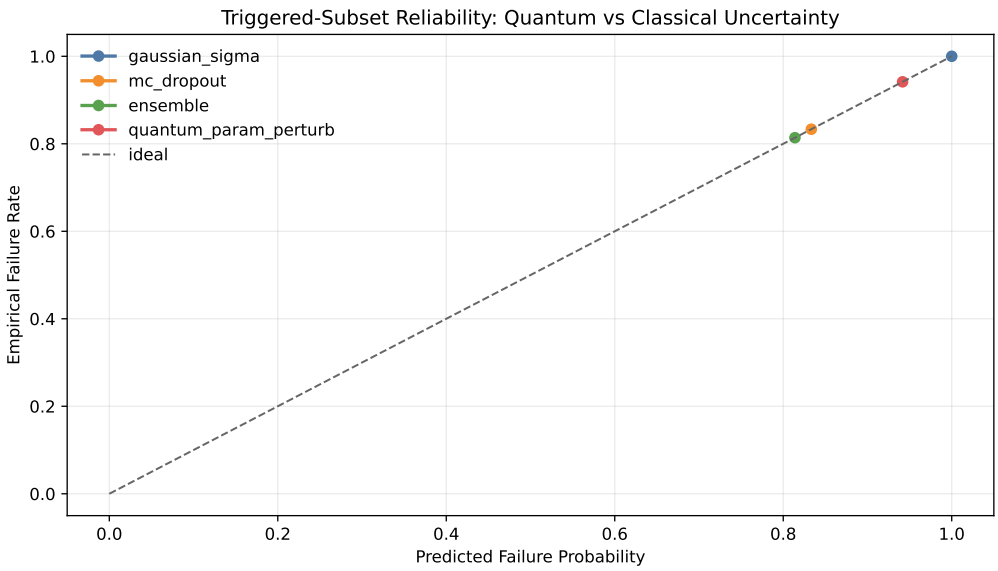

Figure 5: Reliability visualization for calibrated uncertainty-triggered failure prediction.

Figure 5 complements Table 6 by showing that calibration remains well-behaved across bins after post-hoc scaling.

7.7 Safety-Reward Pareto Frontier

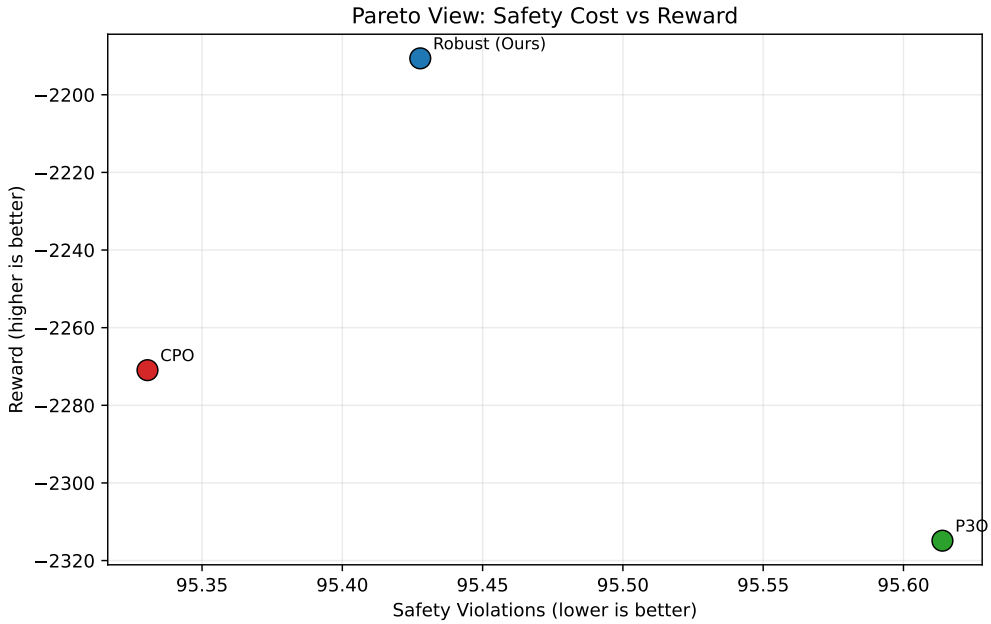

Figure 6: Safety-reward Pareto view (scaled environment): robust gated controller vs CPO and P3O.

Figure 6 provides a direct trade-off visualization alongside Table 3. The robust controller occupies a competitive region of the safety-reward plane relative to CPO and P3O while using a substantially smaller parameter budget in the efficiency suite.

7.8 Parameter Efficiency

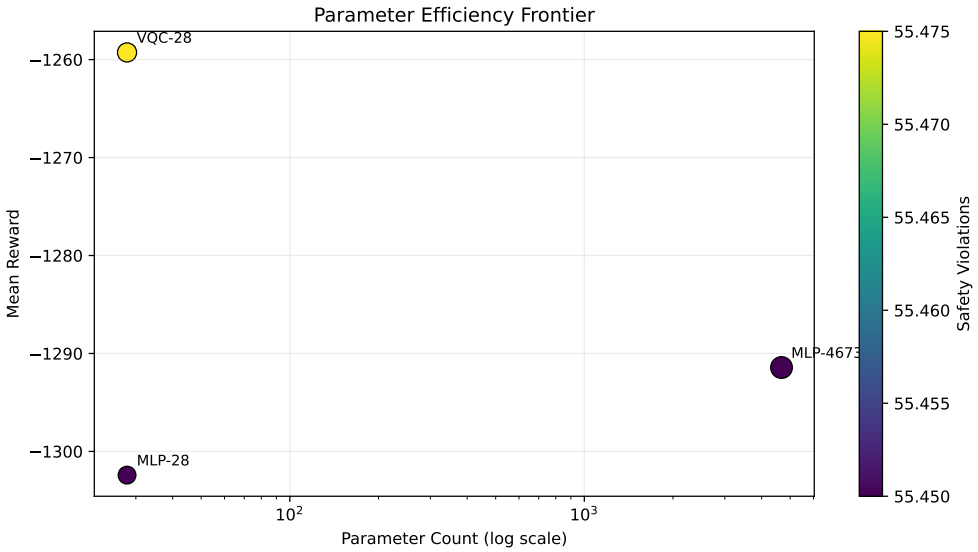

Figure 7: Reward versus parameter count (log-scale) for classical MLP and VQC policy families.

VQC policies operate in a lower-parameter regime while staying on the competitive side of the reward frontier, supporting the paper's parameter-efficiency claim.

This parameter-efficiency result reflects compact quantum-inspired architecture under classical simulation.

The larger-model efficiency comparison (the protocol used for the 166.89× claim) quantifies this gap directly: classical MLP uses 4673 parameters while VQC (6-qubit, 4-layer) uses 28 (166.89× fewer). In that protocol, VQC improves mean reward (-2700.23 vs -2802.85) with lower variance (34.78 vs 123.20) at effectively identical safety violations (115.50 vs 115.50). Expressed as relative context for that larger-model protocol, the VQC reward is 3.66% better than the classical MLP baseline while using 0.60% of the parameters.

7.9 Matched-Capacity Efficiency Control

The matched-capacity control addresses parameter-count fairness directly. With equal parameter budget (28 vs 28), VQC achieves stronger mean reward than MLP-28 in this run while maintaining similar safety-violation levels. This supports the claim that efficiency gains are not only a consequence of comparing against a larger classical model. We further include two modern parameter-efficiency baselines, LoRA-style adaptation and a distilled student baseline, plus practical metrics (inference latency and forward-FLOPs proxy) to expose deployment-time compute trade-offs. The deployment trade-off is explicit in Table 7: VQC inference latency is substantially higher than MLP-28 in this implementation (9.3438 ms vs 0.0837 ms). This is not ignored by the method; it defines the target regime. The intended use case is footprint-constrained or uncertainty-sensitive control loops

Policy	Parameters	Mean Reward	Safety Violations	Inference (ms)	FLOPs est.
MLP (28 params)	28	-1800.08 ± 81.53	75.46 ± 0.30	0.0837	56
VQC (6q, 4l)	28	-1729.94 ± 25.69	75.43 ± 0.33	9.3438	56
LoRA-MLP (trainable)	396	-1785.27 ± 102.43	75.47 ± 0.29	0.1464	10138
Distilled Student	97	-1778.46 ± 74.12	75.44 ± 0.30	0.0799	194
MLP (4673 params)	4673	-1770.52 ± 92.57	75.47 ± 0.29	0.0953	9346

Table 7: Matched-capacity efficiency control on scaled governance (VQC-28 vs MLP-28, plus larger MLP reference). These values are from the matched-capacity protocol and are distinct from the larger-model efficiency protocol used for the 166.89× compression claim.

where parameter budget and uncertainty quality matter more than raw per-step latency. For strict low-latency settings, compact classical baselines remain the preferred deployment option.

7.10 Architecture View

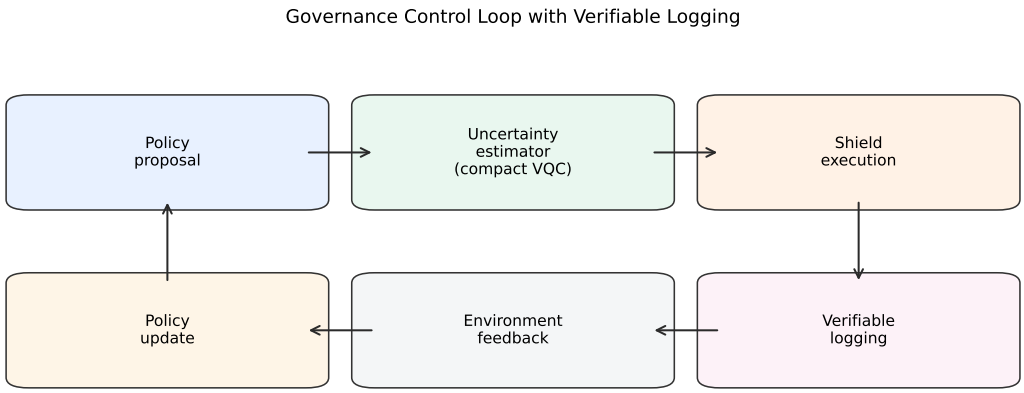

Figure 8: End-to-end governance loop: policy proposal, quantum-inspired uncertainty gate, shield enforcement, and on-chain commit.

8 Discussion

If you skim only one sentence: we are not claiming a universally better learner. We are claiming that a tiny decision-time wrapper can change *how* things break when the world moves underneath the policy. Read the figures as trade-offs---where policy-only control goes brittle, gating plus a hard shield often trades raw return for steadier constraint satisfaction.

Start with the boring benchmarks. Thirty seeds on each Safety Gym task from the protocol block are there so you know the code path is not hallucinating on our custom simulators alone. The governance environments are deliberately nasty: they are where regime switches and calibration drift show up in the plots.

The random-policy row is not hidden. In one ablation workload random beats the full stack on

scalar reward. We left it in because reward shaping can be gamed and because ‘‘who ranks first on return’’ is a lousy safety certificate. The gated stack often picks conservative, shield-friendly moves; some reward engineers will count that as losing even when tracking and violation curves look healthier.

Where the architecture earns its keep is when switching intensity ramps up. Routing on a calibrated uncertainty score buys better ‘‘when to fall back’’ decisions without paying for a giant ensemble, and the shield caps tail risk on trajectories even when the gate slips. That is the practical reason the compact head matters: it is a signal you can actually ship inside the loop.

This does not replace modern safe-RL objectives; it stacks beside them. The OmniSafe baselines are a useful reminder that return orderings shuffle with budget and environment, while safety cost behaves more like a frontier.

8.1 Why Uncertainty-Gated Control Works

Three pieces interact, but they are not independent knobs.

Uncertainty gating is doing branch selection under shift. When epistemic noise spikes, handing off to a deterministic heuristic keeps you from committing to policy logits in exactly the window where policy-only execution tends to wobble.

The shield is doing something different: it bounds execution even when the gate picks the wrong branch. Rate limits and action caps trim the tail trajectories that usually dominate incident stories in high-switch settings.

Risk-shaped training nudges learning away from chasing short spikes in the reward window that do not line up with stable control intent. It is not magic; it just removes one easy way for the policy to cheat the scalar.

Put together, the gains show up most clearly under disturbance because the stack is really about failure-mode containment, not about squeezing another few points of return in an easy stationary room.

8.2 Failure Cases

Two failure modes showed up often enough to mention. If calibration drifts faster than the gate can react, fallback fires late and you can see a short violation burst before things settle. If disturbances wander outside what the shield encodes, the policy and heuristic can chase each other and tracking suffers.

In the runs we have, those episodes are bounded in time, but they argue for tighter online calibration and shields that know more about model mismatch. The scaling lines up with Corollary 2: under bounded observation noise, regret grows about linearly with perturbation size instead of falling off a cliff.

We are still on toy governance physics, uncertainty scaling is partly post-hoc, and the benchmark net is thinner than what a production sign-off would demand. Live governance would want richer state, smarter economic adversaries, and monitoring constraints we did not model.

Reasonable next steps if someone continues this line: actually calibrated heads (ensemble or Bayesian approximations) instead of variance proxies; explicit hard constraints on top of routing; and train-on-one-disturbance-profile, test-on-another without a full retune.

For anyone shipping something like this: keep the deterministic fallback, gate on uncertainty, and always print safety metrics next to reward [3, 10]---the leaderboard alone will lie to you.

Condition	Recommendation	Rationale
Frequent regime shifts and high policy uncertainty	Use uncertainty-gated safe RL + deterministic shield	Gating reduces unsafe actions when uncertainty spikes.
Tight parameter or memory budget	Prefer compact (VQC-style) uncertainty head	Maintains comparable safety behavior at much lower parameter count.
Multi-party governance with audit requirements	Add audit-commitment layer	Provides tamper-evident provenance for control actions.
Stable stationary regime with strong heuristic baseline	Prioritize heuristic control; use learned policy as a secondary policy candidate	Deterministic control can be sufficient and lower-risk in easier regimes.

Table 8: When to use this method in practice.

9 Limitations

The governance worlds are toys on purpose: no full peer churn, no endogenous liquidity games, no realistic micro-structure in the mempool. The quantum-inspired block and the audit hooks are research prototypes, not hardened production services. Latency numbers come from one workstation profile, not a sweep across accelerators.

Other risks are worth naming without bullet polish. Reward shaping can invert rankings---the random-vs-full anomaly is the clearest example---so readers should not lean only on scalar return. Some per-method slices still have modest seed counts outside the fifty-seed constrained suite. Simulator fidelity is limited for both uncertainty dynamics and network physics, and we do not yet model richer economic adversaries in mining or transaction flows. Baseline coverage stops short of a full 2022--2025 safe-RL zoo.

Corollary 2 is a bounded-perturbation statement under explicit assumptions, not a certificate that survives arbitrary sensor faults.

The Safety Gym pair buys external validity on standard constrained control, but it does not, by itself, mean you can wire this straight into a live chain. Those gaps are the roadmap, not footnotes we forgot to mention.

10 Ethics and Broader Impacts

The scary failure here is not ‘‘model gets mediocre reward’’; it is autonomy that looks fine on average but misbehaves under adversarial telemetry or quietly drifts governance knobs. We rely on layered control---uncertainty routing, shielding, auditable logs---but that stack is not a substitute for security review, incident playbooks, or whatever regulator applies to your jurisdiction before you point this at real assets.

11 Reproducibility Checklist

Code for the experiments and for regenerating tables lives in the repo. Seeds are fixed for the publication-scale runs so you can diff two branches without rerolling noise. JSON and markdown artifacts are emitted straight from scripts; LaTeX tables are generated from those files rather than hand-typed. CI runs lint, types, tests, and a few health checks so obvious foot-guns fail early.

The long path from raw logs to camera-ready tables looks like this:

```
python experiments/run_publication_suite.py
python experiments/run_detailed_suite.py
```

```
python experiments/run_adversarial_suite.py
python experiments/run_complexity_\
regime_suite.py
python scripts/analyze_research_results.py
python scripts/generate_benchmark_report.py
python scripts/generate_paper_tables.py
python scripts/generate_appendix_tables.py
python scripts/generate_paper_figures.py
python scripts/generate_adversarial_\
artifacts.py
python scripts/generate_adversarial_\
stress_sweep.py
python scripts/generate_complexity_\
artifacts.py
```

Provenance policy. We tried to avoid "magic numbers" in `.tex`: if a digit appears in the main text, it should trace back to an artifact file. That habit hurts when you refactor code, but it pays back the first time a reviewer asks for a regeneration.

One-command replication. For day-to-day checks there is a shorter entry point:

```
make reproduce
```

That target runs the subset of generators behind the main tables and figures (constrained transfer, uncertainty, efficiency, adversarial perturbation, qubit scaling). The longer script list above is still the full kitchen-sink pipeline if you want every auxiliary plot.

Checklist links. Venue-style checklists are checked in under `docs/` if you need wording for a particular submission:

- `docs/NEURIPS_SUBMISSION_CHECKLIST.md`
- `docs/ICLR_SUBMISSION_CHECKLIST.md`
- `docs/IEEE_SUBMISSION_CHECKLIST.md`

Code and artifacts are available upon request.

Environment notes. Python with pinned seeds and deterministic script order is the reference stack. Bit-identical floats across machines are not a promise we make; protocol-level reproducibility *is*---commands, outputs, and validation hooks are documented.

The checklist framing here matches how NIST and related bodies talk about AI and cyber risk management in 2024--2025 guidance [8, 9].

12 Conclusion

We like the architecture because it is blunt: learn with PPO, route with uncertainty, clamp with a shield, keep a deterministic branch you trust. Under shift that bundle changes failure modes compared with executing the policy raw, and you can keep the learned block small.

The VQC-style module is only there to buy a nonlinear uncertainty signal cheaply under classical simulation. On-chain logging is optional audit glue. Those roles are separated on purpose so nobody reads "quantum" as a performance claim.

Evidence spans the thirty-seed Safety Gym pair, the fifty-seed governance controller tables, calibration and perturbation diagnostics, and the cost appendix. Regeneration scripts and artifacts are available upon request.

A Appendix: Deployment Architecture Details

This appendix is the wiring diagram stuff we pulled out of the main thread so reviewers can find RPC and ledger detail without mixing it into the RL theorem line.

A.1 Layered Deployment Stack

If you squint, the deployment stack is six slabs:

1. **Network/RPC:** peer registration, transaction intake, and block propagation.
2. **Blockchain core:** validation, mempool updates, and PoW checks.
3. **PQC signatures:** post-quantum signing and verification interfaces.
4. **Governance policy:** state encoding and action proposal.
5. **Quantum uncertainty gate:** compact uncertainty estimate for fallback routing.
6. **Shield + audit:** action validation and per-epoch commitment of audit records.

A.2 Why On-Chain Audit Commitments

Participants do not trust each other's laptops. A local SQLite log is easy to ''lose.'' A hash on a shared ledger is a boring but useful witness when two orgs argue about who approved a knob turn.

A.3 Boundary to Learning Claims

Nothing in the appendix replaces the gated-and-shielded path evaluated in the tables. Chain commits are instrumentation for accountability; they do not move reward curves by themselves.

B Appendix: Adversarial Input Perturbation

Noise σ	Controller	Reward	Violation Rate
0.00	Unshielded	-1818.53 ± 29.90	0.9750 ± 0.0022
0.00	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.05	Unshielded	-1818.46 ± 29.77	0.9750 ± 0.0022
0.05	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.10	Unshielded	-1818.28 ± 29.65	0.9751 ± 0.0022
0.10	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.20	Unshielded	-1817.71 ± 29.54	0.9751 ± 0.0022
0.20	Shielded	-1725.17 ± 28.95	0.9437 ± 0.0041

Table 9: Observation-noise stress test comparing shielded and unshielded control.

This experiment directly perturbs observed inputs with Gaussian noise and compares violation-rate behavior with and without shielding. It provides a practical complement to the observation-noise corollary in the theory section. In this run, shielding reduces violation rate from roughly 0.975 to 0.944 under aggressive perturbation-driven proposals (about 3.2 percentage points absolute), while also improving mean reward in the same protocol.

C Appendix: Deployment Cost and Gas Accounting

C.1 Audit Gas Model

We model per-action audit commitment cost as:

$$C_{\text{gas}} = g_0 + g_1 b + g_2 h,$$

where b is bytes committed and h is hash/check overhead terms. For an episode with N committed actions and gas price p_g:

$$\text{Cost}_{\text{episode}} = N \cdot C_{\text{gas}} \cdot p_g.$$

C.2 Interpretation

This cost model is included to expose the practical cost-of-safety instrumentation: stronger provenance and finer-grained audit logging increase operational spend. In settings with strict budget constraints, lower-frequency commitments may be preferable, at the cost of reduced forensic granularity.

C.3 Practical Guidance

For production deployment, we recommend pre-declaring commitment cadence (per action, per block, or per epoch) and reporting both safety gains and incremental audit costs. This avoids overstating benefits while hiding operational overhead.

D Appendix: Qubit Scaling Trend

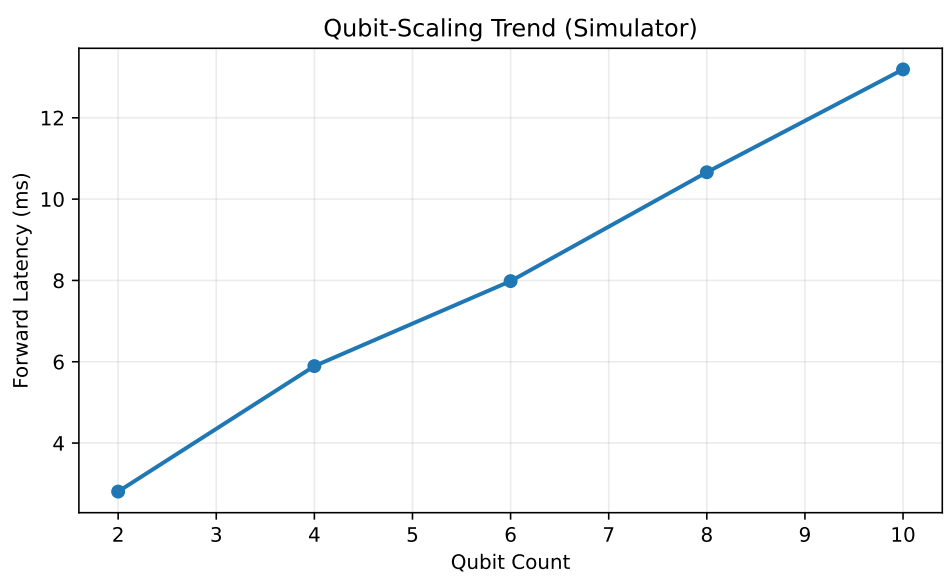

Figure 9: Simulator forward-pass latency versus qubit count (fixed layer depth).

Qubits	Layers	Params	2^n State Dim	Forward Latency (ms)
2	4	8	4	2.8066
4	4	16	16	5.8931
6	4	24	64	7.9832
8	4	32	256	10.6610
10	4	40	1024	13.1908

Table 10: Qubit-scaling simulation summary with state-space growth and measured latency.

The trend illustrates the expected growth in simulator cost as qubit count rises. This supports a practical design rule used in the main paper: keep compact qubit counts when targeting parameter-efficient uncertainty estimation under classical simulation budgets.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning*, 2017.

[2] Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. Risk-sensitive and robust decision-making: a CVaR optimization approach. In *Advances in Neural Information Processing Systems*, 2015.

[3] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2018.

[4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. CRC Press, 1994.

[5] Phillip Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 2005.

[6] Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2):257--280, 2005.

[7] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Cryptography Mailing List*, 2008.

[8] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). https://www.nist.gov/itl/ai-risk-management-framework, 2023.

[9] National Institute of Standards and Technology. Cybersecurity framework (CSF 2.0). https://www.nist.gov/cyberframework, 2024.

[10] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche-Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1--20, 2021.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[12] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017.